Glam Lavish — Inventory Management System

Product Requirements Document (PRD)

Document Version	2.0
Date	March 15, 2026
Status	Revised — 22 development clarifications incorporated
Project	Glam Lavish Inventory & Order Management

---

1. Project Overview

1.1 Background

Glam Lavish is an e-commerce business that currently operates through a WooCommerce website. As order volume grows, there is a need for a dedicated **inventory management system** that works alongside the existing WooCommerce store — providing centralized product management, streamlined order processing, automated courier dispatching, and real-time stock synchronization.

1.2 Goals

- **Centralized Inventory Control** — Manage all products, stock levels, and variations from a single admin panel
- **Bidirectional WooCommerce Sync** — Orders and stock changes flow seamlessly between the inventory system and WooCommerce
- **Automated Order Fulfillment** — Orders automatically dispatch to Steadfast courier upon creation
- **Invoice Generation** — Print thermal receipts (3×4 inch) for each order with QR-based tracking

- **Real-time Order Tracking** — Customers can scan a QR code to track their order status

> **Source of Truth Policy:** WooCommerce is the master for all product content data (name, description, prices, images, categories, variations). The inventory system is the master for stock quantities. Products are never created or content-edited in the inventory system — they are always imported from WooCommerce via webhooks or manual sync. This eliminates duplicate data entry and prevents sync conflicts.

1.3 Success Criteria

- Orders created in the inventory system automatically appear with a Steadfast consignment ID
- WooCommerce orders appear in the inventory system within seconds (via webhooks)
- Stock levels stay synchronized across both platforms
- Invoices print correctly on a thermal printer
- QR code scan opens a tracking page with accurate status

---

2. User Personas

2.1 Admin (Business Owner)

- Full access to all features
- Manages staff accounts (create, edit, reset password, deactivate) — no self-registration
- Views dashboard with business metrics and alert widgets
- Configures WooCommerce and courier API settings

2.2 Staff (Order Processor)

- Creates, edits (before PROCESSING), and manages orders
- Updates order statuses manually
- Prints invoices

- Views product stock levels and adjusts stock
- Cannot manage users or system settings

2.3 Customer (End User)

- Does not log in to the inventory system
- Scans QR code on invoice to track order status via a public tracking page

---

3. Tech Stack

Layer	Technology
Backend	NestJS (TypeScript)
Frontend / Admin Panel	React (TypeScript) + Vite + Tailwind CSS
Database	PostgreSQL + TypeORM
Authentication	JWT (email/password)
Courier Integration	Steadfast API, Pathao API
WooCommerce Integration	WooCommerce REST API v3

---

4. Functional Requirements

4.1 Product Module

> **Design Principle:** Products are managed in WooCommerce and imported into the inventory system as read-only records. The inventory system only manages stock quantities. This eliminates duplicate data entry — staff create/edit products in WooCommerce as they already do, and changes sync automatically.

FR-4.1.1 Product Data (Read-Only, Synced from WooCommerce)

Field	Type	Source	Notes
Product Name	Text	WooCommerce	Read-only in inventory
Short Description	Text	WooCommerce	Read-only
Description	Rich Text	WooCommerce	Read-only
Product Image	URL	WooCommerce	Featured image only (single image, no gallery). Stored as URL, not uploaded locally
Category	Select	WooCommerce	Read-only
Product SKU	Text	WooCommerce	Canonical product identifier
Product Type	Enum	WooCommerce	SIMPLE or VARIABLE
Regular Price	Decimal (BDT)	WooCommerce	Read-only
Sale Price	Decimal (BDT)	WooCommerce	Read-only
wcId	Integer	WooCommerce	Link to WooCommerce product ID
wcPermalink	Text	WooCommerce	"Edit in WooCommerce" link
syncStatus	Enum	Local	SYNCED, PENDING, ERROR

FR-4.1.2 Stock Management (Read-Write, Owned by Inventory System)

Field	Type	Source	Notes
Stock Quantity	Integer	Inventory System	Editable, pushed to WooCommerce
Low Stock Threshold	Integer	Inventory System	Default 5, local-only field

> **Strict Stock Validation:** The system does NOT allow negative stock or backorders. When creating or editing an order, each line item's quantity is validated against

> available stock in real time. If any product has insufficient stock, the order submission is blocked with a clear error message.

FR-4.1.3 Variable Product — Variations (Read-Only + Stock)

Each variation of a variable product contains:

Field	Type	Source	Notes
Attributes	JSON (key-value pairs)	WooCommerce	Read-only. Stores ALL variation attributes dynamically (e.g., Color, Size, Material, Style, Weight — any attribute defined in WC)
Variation SKU	Text	WooCommerce	Read-only
Regular Price	Decimal (BDT)	WooCommerce	Read-only
Sale Price	Decimal (BDT)	WooCommerce	Read-only
Stock Quantity	Integer	Inventory System	Editable, pushed to WooCommerce
Variation Image	URL	WooCommerce	Read-only
wcId	Integer	WooCommerce	Link to WooCommerce variation ID

> **Dynamic Attributes:** Variation attributes are stored as a JSON object (`{ "Color": "Red", "Size": "XL", "Material": "Cotton" }`) rather than fixed columns. This ensures all WooCommerce attributes are synced regardless of what the store defines. The UI renders attribute labels and values dynamically.

FR-4.1.4 Product Import & Sync

- **Initial Import:** "Import All Products" action fetches all WooCommerce products (paginated, 100/page) and creates local records
- **Ongoing Sync:** Products created/updated in WooCommerce sync to the local system via webhooks (content fields only)
- **Stock Sync:** Stock changes in the inventory system are pushed to WooCommerce (`PUT /wc/v3/products/{id}` with `stock_quantity` only)
- **No outbound content sync:** Product content (name, price, description, images) is NEVER pushed from inventory to WooCommerce
- Each product/variation stores a `wcId` to link with WooCommerce

FR-4.1.5 Stock Adjustment

- Staff can manually adjust stock via "Adjust Stock" form on product detail page
- Each adjustment requires a reason (physical count, return, damage, etc.)
- All adjustments logged in `StockAdjustmentLog` for audit trail
- Adjusted stock is automatically pushed to WooCommerce

FR-4.1.6 Product UI Features

- Product list page with search, category filter, stock status badges, low stock alerts
- Product detail page shows all WooCommerce-sourced fields (read-only) + stock adjustment form + stock history
- "Edit in WooCommerce" link opens WC admin in new tab: `{WC_URL}/wp-admin/post.php?post={wcId}&action=edit`
- No product creation or content editing forms in the inventory system

---

4.2 Category Module

> **Design Principle:** Categories are imported from WooCommerce and used as read-only filters in the inventory system. No local creation or editing needed.

Feature	Description
List Categories	Filterable list of all categories (synced from WC)

WC Import	Categories imported alongside products from WooCommerce
WC Webhook Sync	Category changes in WC automatically update local records
Product Filtering	Used to filter products in the product list and order form

---

4.3 Invoice / Order Generator

FR-4.3.1 Order Creation

Field	Type	Required	Notes
Invoice ID	Auto-generated	Yes	Format: `GL-XXXX` (sequential)
Products	Multi-select	Yes	Select product + variation + quantity
Customer Name	Text	Yes	
Customer Phone	Text	Yes	Bangladesh mobile format
Customer Address	Text	Yes	Full delivery address
Shipping Partner	Enum	Yes	Steadfast / Pathao
Shipping Zone	Enum	Yes	Determines shipping fee

FR-4.3.2 Shipping Zones & Fees

Zone	Fee (BDT)
Inside Dhaka	80
Dhaka Sub Area	100
Outside Dhaka	150

FR-4.3.3 Order Creation Flow

1. Staff selects products and quantities (stock validated in real-time)
2. Enters customer details and selects shipping zone

3. System auto-calculates: `Subtotal + Shipping Fee = Grand Total`
4. On submit:
    - Invoice ID generated atomically (GL-0001, GL-0002, ...)
    - Stock decremented for each product/variation
    - **Order automatically pushed to Steadfast courier API**
    - QR code generated linking to public tracking page
    - Stock synced to WooCommerce
5. Order confirmation shown with invoice ID, courier tracking ID, and QR code

FR-4.3.4 Order Statuses

Status	Description
PENDING	Order created, awaiting confirmation
CONFIRMED	Order confirmed and being prepared
PROCESSING	Order being packed/processed
SHIPPED	Handed to courier
DELIVERED	Delivered to customer
CANCELLED	Order cancelled — triggers stock restore + courier cancellation
RETURNED	Order returned by customer — triggers stock restore

> **Cancellation Behavior:** When an order is moved to CANCELLED status:
>
> 1. Stock for all order items is automatically restored (added back) to the respective products/variations
> 2. Restored stock is synced to WooCommerce
> 3. If the order has a Steadfast `consignment_id`, the system calls `POST /api/v1/cancel_order/{consignment_id}` to cancel the courier consignment
> 4. All stock restorations are logged in `StockAdjustmentLog` with reason "Order Cancelled"

> **Return Behavior:** When an order is moved to RETURNED status:
>
> 1. Stock for all order items is automatically restored and synced to WooCommerce
> 2. Stock restorations are logged in `StockAdjustmentLog` with reason "Order Returned"
> 3. Courier consignment is NOT cancelled (already delivered/returned physically)

FR-4.3.5 Order Editing

- Orders can be edited while in **PENDING** or **CONFIRMED** status only
- Once an order reaches **PROCESSING** or beyond, it is locked and cannot be edited
- Editable fields: product line items (add/remove/change quantity), customer name, phone, address, shipping zone
- **Stock auto-adjustment on edit:** When quantities change, the system automatically restores the old quantity and deducts the new quantity, then syncs to WooCommerce
- If the edited order was already pushed to Steadfast, the courier consignment is cancelled and a new one is created with updated details
- Invoice ID does NOT change on edit

FR-4.3.6 Payment Model

> **Full COD Only:** All orders are Cash on Delivery. There is no discount field, no advance payment tracking, and no partial payment support. `Due Amount` always equals `Grand Total` (`Subtotal + Shipping Fee`). The COD amount sent to Steadfast equals the Grand Total.

FR-4.3.7 Invoice Print (3×4 Inch Thermal Receipt)

The invoice prints on a 3-inch × 4-inch thermal receipt with the following layout:

```
┌──────────────────────────┐
│      Glam Lavish         │
│                          │
│ Invoice: GL-0001         │
│ Date: 12/03/2026         │
```

```
│ Courier: Steadfast        │
│ Tracking: 227241927       │
├───────────────────────────┤
│ To: Customer Name         │
│ Ph: 01XXXXXXXXX           │
│ Addr: Full address        │
├───────────────────────────┤
│ Product    Qty    Price   │
│ Item 1      1   1,499.00│
├───────────────────────────┤
│ Subtotal:       1,499.00│
│ Delivery:          80.00│
│ Grand Total:   1,579.00 │
│ Due Amount:    1,579.00  │
│                           │
│        [QR Code]          │
└───────────────────────────┘
```

- Uses CSS `@page { size: 3in 4in; margin: 2mm; }` for thermal printing
- `react-to-print` library triggers the browser print dialog
- QR code (~2cm × 2cm) at the bottom encodes the tracking URL

4.4 QR Code & Order Tracking

FR-4.4.1 QR Code Generation

- Generated at order creation time using the `qrcode` npm package
- Encodes URL: `{FRONTEND_URL}/tracking/{invoiceId}`
- Stored as a data URL on the Order record
- Rendered on the invoice and in the order detail page

FR-4.4.2 Public Tracking Page

- **No authentication required** — accessible by anyone with the URL/QR code
- Displays:
  - Order status timeline (visual progress: Pending → Confirmed → Shipped → Delivered)
  - Invoice ID and order date
  - Customer name (partially masked for privacy)
  - Courier name and tracking ID
  - Link to courier's own tracking page (if available)

4.5 WooCommerce Integration

FR-4.5.1 Configuration

Setting	Description
WooCommerce Store URL	The WordPress/WooCommerce site URL
Consumer Key	WC REST API consumer key (Read/Write)
Consumer Secret	WC REST API consumer secret
Webhook Secret	For verifying incoming webhook signatures

> **Setup Note:** Generate API keys from WooCommerce → Settings → Advanced → REST API → Add Key. Select "Read/Write" permissions.

FR-4.5.2 Inbound Sync (WooCommerce → Inventory) — Product Content + Orders

Webhook Event	Action
`order.created`	Create local Order + OrderItems, decrement local stock (do NOT push stock back to WC — WC already decremented), auto-push to Steadfast courier
`order.updated`	Update local order status and details
`product.created`	Create local Product with variations (all content fields)
`product.updated`	Update local product **content fields only** (name, description, prices, images, category) — do NOT overwrite stock
`product.deleted`	**Soft-delete** the local product (set `deletedAt` timestamp). Product disappears from product list but order history referencing it is preserved

- Webhooks verified via `X-WC-Webhook-Signature` (HMAC-SHA256)
- Matched to local records via `wcId` / `wcOrderId`
- Stock is NEVER updated from inbound product webhooks (inventory system owns stock)

WooCommerce Order Status Mapping:

WC Status	Local OrderStatus
`pending`	PENDING
`on-hold`	PENDING
`processing`	PROCESSING
`completed`	DELIVERED
`cancelled`	CANCELLED

`refunded`	RETURNED
`failed`	CANCELLED

WC Order Shipping Zone Parsing:

When an order arrives via WC webhook, the shipping zone is determined from the WC order's shipping method/zone data (not parsed from address text). The system reads the `shipping_lines` from the WC order payload and uses the WC-assigned shipping cost directly. If no shipping data is available, defaults to "Outside Dhaka" (150 BDT).

FR-4.5.3 Outbound Sync (Inventory → WooCommerce) — Stock Only

> **Important:** Outbound sync is limited to stock quantity updates. Product content (name, price, description, images) is NEVER pushed from the inventory system to WooCommerce.

Event	API Call
Local order created	`PUT /wc/v3/products/{id}` — update `stock_quantity` only
Manual stock adjustment	`PUT /wc/v3/products/{id}` — update `stock_quantity` only
Variable product stock change	`PUT /wc/v3/products/{id}/variations/{vid}` — update `stock_quantity` only

FR-4.5.4 Deduplication

- When outbound stock sync triggers a WC `product.updated` webhook back, the system checks `wcLastSyncedAt` timestamp
- If the entity was synced within the last 5 seconds, the inbound webhook is skipped
- `wcLastSyncedAt` is tracked per entity (not global) — syncing product A does not block a legitimate webhook for product B
- All sync operations logged in `SyncLog` table

FR-4.5.5 Manual Sync

- "Sync Now" button in Settings triggers a full recovery sync:
  - **Inbound:** Fetches all WC products (paginated, 100/page) and upserts content fields locally (not stock)
  - **Inbound:** Fetches recent WC orders (last 30 days) and upserts locally
  - **Outbound:** Pushes all local stock values to WooCommerce (local stock wins)
- Sync progress and results displayed in a sync log viewer

FR-4.5.6 Stock Reconciliation Cron

- Runs every hour
- Compares local stock vs WooCommerce stock for all products
- If mismatch detected, local stock wins — pushed to WC
- Discrepancies logged in SyncLog for admin review
- Prevents silent stock drift from network failures or missed webhooks

---

4.6 Courier Integration — Steadfast

FR-4.6.1 Auto-Push on Order Create

When an order is created — **whether manually in the inventory system or received via WooCommerce webhook** — it is **automatically** pushed to Steadfast:

Detail	Value
API Endpoint	`POST https://portal.steadfast.com.bd/api/v1/create_order`
Auth Headers	`Api-Key`, `Secret-Key`
Payload	`invoice`, `recipient_name`, `recipient_phone`, `recipient_address`, `cod_amount`
Response	Returns `consignment_id` and `tracking_code`

- `consignment_id` and `tracking_code` stored on the Order record
- If Steadfast API fails, the order is still created but flagged with `courierConsignmentId = null`

- A "Retry Courier" button appears in the order detail for manual retry

FR-4.6.2 Manual Status Updates (No Auto-Polling)

> **No automated status polling.** Order statuses are updated manually by staff from the order detail page. There is no cron job polling Steadfast for delivery status updates. Staff selects the new status from a dropdown on the order detail page.

FR-4.6.3 Consignment Cancellation

- When an order is **CANCELLED** and has a valid `consignment_id`, the system calls Steadfast's cancellation endpoint
- API: `POST https://portal.steadfast.com.bd/api/v1/cancel_order/{consignment_id}`
- If the cancellation API fails, the order is still cancelled locally but a warning is shown to staff
- When an order is **edited** (before PROCESSING), the old consignment is cancelled and a new one is created with updated details

FR-4.6.4 Pathao Courier (Future)

- OAuth2 token-based authentication
- Order creation via `POST /aladdin/api/v1/orders`
- Similar status polling mechanism
- Implemented in Phase 6

4.7 Dashboard

Widget	Description
Total Orders Today	Count of orders created today
Revenue Today	Sum of `grandTotal` for today's orders
Pending Orders	Count of orders in PENDING status
Low Stock Alerts	Products with stock below threshold (default 5 per product)
Failed Courier Pushes	Orders where Steadfast push failed (`courierConsignmentId = null`). Shows count with link to filtered order list
Sync Errors	Recent WooCommerce sync failures from `SyncLog` . Shows count and last error timestamp
Recent Orders	Table showing last 10 orders with status badges

> **Dashboard Alerts:** The dashboard is the primary notification mechanism. No email or push notifications are sent. Staff should check the dashboard regularly for failed courier pushes, sync errors, and low stock alerts.

---

5. Database Schema Overview

5.1 Entity Relationship Summary

```
User ──< Order ──< OrderItem >── Product
                                    │
                    ProductVariation ┘
                          │
Category ──< Product      │
                          │
Order ──< SyncLog         │
```

```
                          |
InvoiceCounter (singleton)
```

5.2 Key Entities

Entity	Key Fields	Notes
User	id, email, password, name, role	Roles: ADMIN, STAFF. Admin-created only, no self-registration
RefreshToken	id, userId, token, expiresAt, isRevoked	Stores refresh tokens for JWT auth. 7-day expiry
Category	id, name, slug, wcId	WC-linked, read-only
Product	id, name, sku, type, imageUrl, prices, stockQuantity, lowStockThreshold, wcId, wcPermalink, syncStatus, wcLastSyncedAt	Soft delete, content from WC, stock locally managed. Single featured image only
ProductVariation	id, productId, sku, attributes (JSON), prices, stock, imageUrl, wcId, wcLastSyncedAt	`attributes` is a JSON column storing all variation attributes dynamically (e.g., `{"Color": "Red", "Size": "XL"}`)
StockAdjustmentLog	id, productId, variationId, previousQty, newQty, reason, adjustedBy, createdAt	Audit trail. Reasons include: manual adjustment, order created, order cancelled, order returned, order edited
Order	id, invoiceId, source, status, customer info, shippingZone, shippingFee, subtotal, grandTotal, courier info, wcOrderId, wcShippingCost	Central entity. `grandTotal = subtotal + shippingFee`. Full COD only
OrderItem	id, orderId, productId, variationId, quantity, unitPrice, totalPrice	Line items
SyncLog	id, direction, entityType, entityId, status, payload, error, createdAt	Audit trail for all WC sync operations

InvoiceCounter	id (singleton), lastNum	Atomic ID generation. Format: GL-XXXX. No yearly reset, increments indefinitely (GL-10000+ allowed)

5.3 TypeORM Implementation Notes

- All entities use UUID primary keys
- Money fields: `decimal` type with `precision: 10, scale: 2`
- Enums: stored as PostgreSQL enum types
- Soft delete on Product via `@DeleteDateColumn()` — triggered by WC `product.deleted` webhook
- ProductVariation `attributes` field: `jsonb` column type in PostgreSQL for efficient querying
- Invoice counter uses `QueryRunner` with `SELECT ... FOR UPDATE` row locking
- Timestamps: `createdAt` and `updatedAt` on all entities via `@CreateDateColumn()` / `@UpdateDateColumn()`
- All timestamps stored in UTC in the database; converted to BST (UTC+6) on the frontend

---

6. API Endpoints Overview

6.1 Authentication

Method	Endpoint	Description
POST	`/api/auth/login`	Login with email/password, returns access token (1h) + refresh token (7 days)
POST	`/api/auth/refresh`	Exchange a valid refresh token for a new access token + refresh token pair
POST	`/api/auth/logout`	Revoke the current refresh token (invalidates session)
GET	`/api/auth/me`	Get current authenticated user

6.2 Products

> Products are read-only (synced from WooCommerce). Only stock-related endpoints are writable.

Method	Endpoint	Description
GET	`/api/products`	List products (paginated, filterable by category, stock status, search)
GET	`/api/products/:id`	Product detail with variations and stock history
PATCH	`/api/products/:id/stock`	Adjust stock quantity (body: `{ quantity, reason }`) — pushes to WC
PATCH	`/api/products/variations/:id/stock`	Adjust variation stock (body: `{ quantity, reason }`) — pushes to WC
GET	`/api/products/:id/stock-history`	View stock adjustment log for a product

6.3 Categories

> Categories are read-only (synced from WooCommerce).

Method	Endpoint	Description
GET	`/api/categories`	List all categories (for product filtering)

6.4 Orders

Method	Endpoint	Description
GET	`/api/orders`	List orders (filterable by status, source, date range). Supports `?startDate=&endDate=&status=&source=&page=&limit=`

GET	`/api/orders/export`	Export filtered orders as CSV. Same query params as list endpoint
GET	`/api/orders/:id`	Order detail with items
POST	`/api/orders`	Create order (auto Steadfast push + stock decrement + WC stock sync)
PATCH	`/api/orders/:id`	Edit order (only PENDING/CONFIRMED). Auto-adjusts stock, re-pushes to Steadfast
PATCH	`/api/orders/:id/status`	Update order status. CANCELLED triggers stock restore + courier cancel. RETURNED triggers stock restore
GET	`/api/orders/:id/invoice`	Invoice print data
GET	`/api/orders/:id/qr`	QR code image

6.5 User Management (Admin Only)

Method	Endpoint	Description
GET	`/api/users`	List all users (admin only)
POST	`/api/users`	Create a new user account (admin only). Body: `{ email, password, name, role }`
PATCH	`/api/users/:id`	Update user details (admin only). Admin can reset password here
DELETE	`/api/users/:id`	Deactivate a user account (admin only)

> **No self-registration.** Users cannot create their own accounts. Only admins can create staff accounts and reset passwords.

6.6 Public Tracking

Method	Endpoint	Description
GET	`/api/tracking/:invoiceId`	Public order tracking (no auth)

6.7 WooCommerce

Method	Endpoint	Description
POST	`/api/woocommerce/webhook/order`	Receive WC order webhook (creates local order + auto-pushes to Steadfast)
POST	`/api/woocommerce/webhook/product`	Receive WC product webhook (create/update/delete)
POST	`/api/woocommerce/import/products`	Import all products from WooCommerce (initial setup + recovery)
POST	`/api/woocommerce/sync/products`	Manual full product sync (content from WC, stock pushed to WC)
POST	`/api/woocommerce/sync/orders`	Manual full order sync
GET	`/api/woocommerce/sync-logs`	View sync history

6.8 Dashboard

Method	Endpoint	Description
GET	`/api/dashboard/stats`	Summary statistics
GET	`/api/dashboard/low-stock`	Low stock product alerts
GET	`/api/dashboard/recent-orders`	Last 10 orders

7. Frontend Pages

Route	Page	Auth	Description
`/login`	Login	Public	Email/password login form
`/`	Dashboard	Protected	Stats, recent orders, low-stock alerts, failed courier alerts, sync error alerts
`/products`	Product List	Protected	Searchable, filterable product table with stock badges and low stock alerts. **Traditional pagination, 25 items per page**
`/products/:id`	Product Detail	Protected	Read-only product info (from WC) with dynamic variation attributes + stock adjustment form + stock history + "Edit in WooCommerce" link
`/orders`	Order List	Protected	Filterable by status, source, **date range**. **"Download CSV" export button** for filtered results. **Traditional pagination, 25 items per page**
`/orders/new`	Order Form	Protected	**Main order generator** with product search, line items, customer info, shipping calculator. Real-time stock validation (blocks if insufficient)
`/orders/:id`	Order Detail	Protected	View order, **edit order** (if PENDING/CONFIRMED), update status, courier info, retry courier
`/orders/:id/invoice`	Invoice Print	Protected	3×4 thermal receipt, print button
`/tracking/:invoiceId`	Tracking	**Public**	Order status timeline, courier tracking
`/settings`	Settings	Admin only	WC API config, user management (create/edit/reset password/deactivate users)

UI Notes:

- All dates/times displayed in **Bangladesh Standard Time (UTC+6)**
- UI is **English only** — no Bangla or multi-language support
- JWT access tokens auto-refresh silently using refresh tokens — users stay logged in for up to 7 days without re-entering credentials

---

8. Non-Functional Requirements

8.1 Performance

- Order creation (including Steadfast push) completes within 3 seconds
- Product listing loads within 1 second for up to 1,000 products
- WooCommerce webhook processing completes within 2 seconds

8.2 Security

- All API endpoints (except tracking and WC webhooks) require JWT authentication
- **JWT Access Token:** 1-hour expiry. Sent as `Authorization: Bearer <token>` header
- **JWT Refresh Token:** 7-day expiry. Stored in `RefreshToken` table. Used to silently obtain new access tokens
- Refresh tokens are revoked on logout and can be revoked by admin
- Passwords hashed with bcrypt (minimum 10 salt rounds)
- **No self-registration or password reset.** Admins create and manage all user accounts
- WooCommerce webhooks verified via HMAC-SHA256 signature
- Steadfast/Pathao API keys stored as environment variables, never in code
- Input validation on all endpoints using `class-validator`

8.3 Reliability

- Order creation uses database transactions — if any step fails, everything rolls back

- Courier API failures do not block order creation — orders are flagged for retry
- WooCommerce sync failures are logged and can be retried manually
- Invoice counter uses row-level locking to prevent duplicate IDs under concurrent requests

8.4 Localization

- **Language:** English only — no multi-language or Bangla support
- **Timezone:** All dates and times displayed in **Bangladesh Standard Time (BST, UTC+6)**
- **Server storage:** All timestamps stored in UTC in the database
- **Frontend conversion:** Dates converted to BST on the frontend before rendering
- **Currency:** Bangladesh Taka (BDT) only

8.5 Scalability

- Designed for a single-store operation (Glam Lavish)
- Can handle up to 500 orders/day comfortably
- Database indexes on frequently queried fields (invoiceId, wcId, status, createdAt)

---

9. Delivery Timeline

Phase	Duration	Deliverables
Phase 1: Scaffolding	Day 1–2	Project setup, database schema (including StockAdjustmentLog, RefreshToken, dynamic attributes), auth module with JWT refresh tokens, user management (admin creates accounts)
Phase 2: Products + WC Import	Day 3–4	WC product import service, product list (read-only, 25/page pagination), product detail with dynamic variation attributes + stock adjustment, categories import
Phase 3: Orders + Steadfast	Day 5–8	Order creation with strict stock validation, auto Steadfast push (all orders), QR code, order editing (before PROCESSING), cancellation (stock restore + courier cancel), return stock restore, order pages with date filter + CSV export

Phase 4: Invoice + Tracking	Day 9–10	Thermal invoice print, public tracking page
Phase 5: WooCommerce Sync	Day 11–13	Product/order webhooks (including product.deleted), WC order status mapping, WC shipping zone parsing, stock push, dedup, manual sync, sync logs
Phase 6: Dashboard + Polish	Day 14–16	Dashboard with alert widgets (failed couriers, sync errors), stock reconciliation cron, Pathao integration, UX polish

---

10. Risks & Mitigations

Risk	Impact	Mitigation
Steadfast API downtime	Orders created without courier dispatch	Retry mechanism + manual dispatch button + dashboard alert for failed pushes
Steadfast cancel API failure	Cancelled order still has active courier consignment	Warning shown to staff, manual cancellation in Steadfast portal
WooCommerce webhook failures	Stock desync between platforms	Manual "Sync Now" button + sync log monitoring + hourly reconciliation cron
Concurrent invoice ID generation	Duplicate invoice numbers	Row-level locking with `SELECT ... FOR UPDATE`
WC sync loops (outbound triggers inbound webhook)	Infinite sync cycle	5-second deduplication window using per-entity `wcLastSyncedAt`
Stock drift from network failures	Local and WC stock diverge silently	Hourly stock reconciliation cron (local wins) + SyncLog error alerts on dashboard
Simultaneous orders from WC + inventory	Double stock decrement	Both decrements are valid (different orders) — no conflict, final stock = original - both quantities
Order edit after courier push	Stale consignment in Steadfast	Old consignment cancelled, new one created automatically on edit

Negative stock attempt	Data integrity issue	Strict validation blocks orders with insufficient stock at API level

---

11. Future Enhancements

- Pathao courier full integration (Phase 6)
- Bulk order import/export (CSV)
- Product barcode scanning
- Customer database with order history
- SMS notifications for order status updates
- Multi-store support
- Financial reports and analytics
- Mobile app for order management

This document serves as the primary reference for the Glam Lavish Inventory Management System development. Requirements may evolve based on use-case feedback during development.